

Adaptive, Intelligent Multi-Agent System for Autonomous Software Planning, Code Generation, and Self-Debugging

Project ID R26-SE-029

Project Proposal Report

Student name – Student ID

Dilakshan P – IT22297372

B.Sc. (Hons) Degree in Information Technology Specialized in
Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Dilakshan P	IT22297372	P. Dilakshan

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Ms. Sanjeevi Chandrasiri	2026/03/20	Sanjeevi

ABSTRACT

Modern software development still requires substantial manual effort for backend RESTful API implementation. Although AI coding agents such as Cursor IDE, Replit Agent, and Claude Code can generate full projects, they typically rely on cloud-hosted models and produce code that does not follow a developer’s personal coding conventions. This mismatch often requires extensive manual refactoring and increases technical debt. This research proposes a Style-Aware Generative Coding Agent integrated into a Human-in-the-Loop multi-agent system for automated backend synthesis. The system analyzes a developer’s existing repositories using AST parsing with Tree-sitter and applies Retrieval-Augmented Generation (RAG) to extract stylistic patterns such as naming conventions, middleware usage, and error-handling strategies. These conventions are then enforced during multi-file backend generation.

A key architectural contribution is a multi-tenant RAG pipeline using per-user namespaced collections in ChromaDB. Personalization is achieved through inference-time context injection without model fine-tuning, enabling multiple developers to share a single cloud LLM while preventing cross-user style leakage. The system integrates four mechanisms: (1) RAG-augmented prompt engineering for style injection, (2) episodic memory-driven logic reuse inspired by Reflexion-style agents, (3) pre-testing syntax and logic validation through iterative self-editing, and (4) scalable multi-file project generation via a cloud-based file manager using object storage. The implementation leverages FastAPI, LangGraph, and containerized execution environments.

Keywords: Style-Aware Code Generation, Full-Project Synthesis, Multi-Tenant RAG, Episodic Memory, Node.js Backend Automation, Multi-Agent Systems, Cloud API

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported me throughout the completion of this 4th year research project. First and foremost, I extend my deepest thanks to my supervisors, Ms. Sanjeevi Chandrasiri and Ms. Karthiga Rajendran, whose continuous guidance, expert advice, and encouragement were instrumental at every stage of this research. Their insightful feedback on system architecture, methodology, and literature shaped the quality of this work.

I would also like to extend my appreciation to the CDAP panel for their constructive evaluations and recommendations during each progress review. Their critical perspectives helped strengthen both the technical and academic rigour of this proposal.

I am grateful to my group members within the Multi-Agent Framework project for their teamwork, collaborative discussions, and mutual support throughout the development process. Their individual contributions to the Planning, Testing, and Orchestration components have directly informed and enriched the design of this Coding Agent component.

Finally, I express my heartfelt gratitude to my parents and friends for their unwavering encouragement, patience, and moral support throughout this academic journey.

TABLE OF CONTENT

Table of Contents

DECLARATION	2
ABSTRACT	3
ACKNOWLEDGEMENT	3
TABLE OF CONTENT	4
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ABBREVIATIONS	7
1. INTRODUCTION	8
2. OBJECTIVE	12
3. METHODOLOGY	13
4. HIGH-LEVEL SYSTEM ARCHITECTURE	16
6. COMMERCIALIZATION PLAN	18
7. BUDGET AND JUSTIFICATION	20
8. WORK BREAKDOWN STRUCTURE (WBS)	20
9. GANTT CHART	21

LIST OF TABLES

Table 3.3: Tools and Platforms in Tabular format

Table 7.1: Budget and Justification Format

LIST OF FIGURES

Figure 4.1: System Architecture Diagram

Figure 4.2: Context-Aware Planning Agent Sub-Diagram

Figure 4.2: Context-Aware Planning Agent Sub-Diagram

Figure 8.1: Work Breakdown Structure

Figure 9.2: Gantt Chart

LIST OF ABBREVIATIONS

RAG – Retrieval-Augmented Generation

AST – Abstract Syntax Tree

LLM – Large Language Model

FSM – File System Manager

CoT – Chain-of-Thought

API – Application Programming Interface

S3 – Simple Storage Service (AWS)

GCS – Google Cloud Storage

vLLM – Virtual Large Language Model serving framework

1. INTRODUCTION

1.1 Background and Context

Software development, and in particular the construction of backend RESTful APIs using Node.js/Express and MongoDB, remains a deeply labour-intensive discipline despite significant advances in AI-powered coding assistants. Developers in practice spend a substantial proportion of their working time on repetitive boilerplate generation — creating models, controllers, routes, middleware, and documentation — while simultaneously striving to maintain a coherent personal or team-defined coding style. This style is not merely cosmetic. It encodes years of accumulated engineering judgement: camelCase field naming, consistent try-catch error propagation, standardised JWT authentication middleware, Mongoose schema conventions, and project-specific architectural layering. Deviations from these conventions introduce integration friction, increase code review burden, and erode long-term maintainability.

The emergence of commercial AI coding agents — including Cursor Composer, GitHub Copilot Workspace, Replit Agent, Aider, and Claude Code — has partially mechanised project-level code generation. However, these tools share a fundamental architectural constraint: they are style-agnostic systems with no access to the developer's established conventions. They generate syntactically valid, functionally reasonable code, but the output systematically diverges from the developer's personal style. Researchers studying professional developers using AI coding tools consistently observed that generated code required significant post-hoc refactoring to align with team conventions (Ghai et al., 2024; Suri et al., 2023). For backend engineers maintaining multiple concurrent client projects, this refactoring can consume 30–40% of total development time.

In the Sri Lankan IT context, this problem is particularly acute. Many small-to-medium software companies and freelance developers simultaneously maintain 5–10 client projects, each with distinct style requirements negotiated over months or years of client engagement. Rather than requiring developers to install and manage local AI infrastructure, the proposed system is exposed as a cloud-based REST API — accessible from any IDE, CLI client, or CI/CD pipeline with no local setup overhead. This lowers the barrier to adoption significantly for Sri Lanka's growing software development community, directly contributing to SDG 9 (Industry, Innovation and Infrastructure).

This research addresses the core problem through a Generative Coding Agent and Module Synthesis component embedded within a broader Human-in-the-Loop Multi-Agent System. The agent introduces Style-Aware Full-Project Synthesis: it analyses the developer's existing repository using AST parsing and Retrieval-Augmented Generation to extract personal coding conventions, and enforces these conventions at inference time through a multi-tenant namespaced RAG architecture. Every developer's style data is stored in a fully isolated ChromaDB collection on the cloud backend, meaning a single shared language model can produce personalised, style-consistent code for many different developers concurrently — a capability absent from all existing tools.

1.2 Literature Review

Multi-File and Long-Horizon Code Generation A foundational challenge in full-project backend generation is maintaining coherence across multiple interdependent files without exhausting the finite context window of the underlying language model. Holt et al. (2024) address this directly with L2MAC — the first practical LLM-based stored-program automatic computer — which implements a von Neumann-style architecture comprising a persistent file store and a Control Unit that manages discrete read and write operations across an entire codebase [1]. By decoupling file management from the model's context window, L2MAC enables unbounded multi-file generation and substantially outperforms context-window limited baselines on system design tasks. This work provides the direct architectural template for the File System Manager component of the proposed agent, adapted here for cloud object storage rather than local disk.

Repository-Level Context as a Personalization Signal While L2MAC solves the problem of scale, it does not address stylistic fidelity to a specific developer's conventions. The foundational insight that repository context can serve as a powerful personalisation signal comes from Shrivastava et al. (2023a), who introduced Repository-Level Prompt Generation (RLPG) — a framework that constructs example-specific prompts by retrieving contextual information from an entire codebase including imports, sibling files, and parent class definitions [3]. Without modifying model weights, RLPG achieves a 36% relative improvement over Codex on single-line completion tasks, establishing that inference-time context injection is a viable and effective personalisation strategy. Shrivastava et al. (2023b) extended this idea to training with RepoFusion, demonstrating that models trained with repository context significantly outperform models roughly 70 times larger on completion benchmarks [2]. Taken together, these works establish the theoretical foundation for this system's core claim: that a shared cloud model can produce deeply personalised outputs when supplied with the right per-user context. The benchmark established by Ding et al. (2023) in CrossCodeEval further validates this direction, providing the first evaluation framework that explicitly requires cross-file contextual understanding for correct code completion across multiple programming languages [9].

Iterative Self-Correction and the Role of Verbal Feedback Generating a correct first draft is rarely sufficient for complex backend modules. The question of how AI agents learn from their own mistakes without expensive model retraining has been addressed most compellingly by Shinn et al. (2023) through Reflexion — a framework in which agents reflect verbally on task failures, store these reflections in an episodic memory buffer, and incorporate them into subsequent generation attempts [4]. Achieving 91% pass@1 on HumanEval without any weight updates, Reflexion demonstrates that persistent verbal memory can serve as a lightweight but powerful substitute for gradient-based learning. The episodic memory mechanism of the proposed system is directly modelled on this approach: after each generation cycle, style violations are recorded as verbal reflections and retrieved in future prompts, enabling the agent to avoid repeating the same style errors across sessions. Chen et al. (2023) complement this with Self-Debug, showing that LLMs can identify and correct their own errors through a rubber-duck debugging process — explaining generated code in natural language to surface logical inconsistencies — achieving up to 12%

improvement on TransCoder and MBPP without any external error messages [5]. This technique forms the final stage of the proposed system's pre-testing validation pipeline.

Fault-Aware Editing and Execution-Grounded Validation Verbal self-reflection alone is not sufficient to guarantee syntactic and logical correctness before code reaches the Testing Agent. Zhang et al. (2023) address this with Self-Edit, a fault-aware code editor that wraps execution results as supplementary comments and uses them to guide targeted refinement of the generated output [6]. Tested across nine LLMs ranging from 110M to 175B parameters, Self-Edit improves pass@1 by 89% on APPS-dev and 48% on HumanEval — demonstrating that execution-grounded feedback is dramatically more effective than generation alone. The proposed system's Pre-Testing Validation Layer directly implements this generate-and-edit paradigm, using esprima for AST-level syntax checking and ephemeral Docker containers for mock execution before handover. Yang et al. (2023) provide the standardised framework for this kind of interactive execution-grounded evaluation through InterCode, which treats code generation as a reinforcement learning environment with code as actions and execution feedback as observations, using Docker-sandboxed environments for safe and reproducible testing [10]. The ephemeral container design of the proposed validation layer follows InterCode's isolation model directly.

Modular Generation and Documentation-Grounded Retrieval Complex backend projects consist of logically interdependent modules, and generating them as monolithic blocks is both brittle and hard to debug. Le et al. (2024) tackle this with CodeChain, which introduces a chain of self-revisions guided by representative sub-modules extracted and clustered from previous generation iterations [8]. By selecting cluster centroids as canonical reusable implementations, CodeChain achieves a 35% relative improvement on APPS and 76% on CodeContests, demonstrating that intelligent reuse of verified sub-modules is a more effective strategy than generating each module from scratch. The Logic Injection mechanism of the proposed system implements this clustering approach: verified sub-modules such as JWT middleware, Mongoose error handlers, and pagination utilities are extracted, embedded, clustered, and stored in the user's episodic memory for reuse across future projects. For cases where the developer provides no existing repository, Zhou et al. (2023) establish the complementary approach through DocPrompting — retrieving relevant API documentation before generation to handle unseen library functions, improving CodeT5 by 52% in relative pass@1 gain on CoNaLa [7]. This motivates the curated Mongoose, Express, and JWT documentation collection used as fallback RAG context in the proposed system.

Retrieval-Augmented Generation as the Unifying Framework The common thread running through RLPG, RepoFusion, and DocPrompting is the principle established by Lewis et al. (2020) in the original RAG paper: that grounding language model outputs in retrieved external knowledge substantially improves factual accuracy, specificity, and relevance without modifying model weights [11]. The proposed system applies this principle in a novel multi-tenant configuration: rather than retrieving from a single shared knowledge base, each developer's personal style data occupies its own isolated namespace in ChromaDB, and retrieval at generation time is scoped exclusively to that namespace. This transforms RAG from a knowledge-grounding mechanism into a personalisation mechanism — a distinction that represents the core architectural contribution of this research.

1.3 Research Gap

Style-Agnostic Commercial Tools The commercial tools that currently dominate AI-assisted backend development — Cursor, Replit Agent, Claude Code, and GitHub Copilot Workspace — share a fundamental limitation: they treat all developers identically. Despite their capability for full-project generation, none implement any mechanism for extracting, learning, or enforcing a specific developer's personal coding conventions. As a result, the generated code systematically diverges from the developer's established patterns, requiring substantial manual refactoring that erodes the productivity gains these tools are designed to provide (Ghai et al., 2024; Suri et al., 2023). This gap is not a minor inconvenience — for developers maintaining multiple concurrent client projects with distinct style requirements, it can consume 30–40% of total development time.

The Missing Integration in Academic Research Academic research has addressed individual aspects of the problem in isolation, but no existing work integrates them into a cohesive system. L2MAC solves the problem of generating coherent multi-file codebases at scale [1], but it has no mechanism for stylistic fidelity to a specific developer's conventions. Reflexion and Self-Debug provide powerful self-correction capabilities [4][5], but they operate at the level of functional correctness — passing unit tests — not stylistic consistency. RLPG and RepoFusion demonstrate the power of repository-level context as a generation signal [2][3], but they support single-file completion rather than full-project multi-file synthesis, and they do not persist learned conventions across projects or users. CodeChain introduces modular self-revision [8], but without any mechanism for anchoring the generated modules to a specific developer's stylistic preferences. The result is that while the theoretical building blocks exist across the literature, no system has assembled them into a practical, cloud-deployed framework capable of learning and enforcing a developer's personal style persistently across projects.

The Multi-Tenant Challenge of Cloud Deployment An additional gap emerges specifically in the cloud deployment context. A naive cloud architecture using a single shared language model cannot provide personalised outputs for different developers without either maintaining separate fine-tuned model instances per user — which is prohibitively expensive at scale — or accepting the style-agnostic limitation of all commercial tools. The proposed system resolves this tension through a multi-tenant namespaced RAG architecture: each developer's style data occupies an isolated ChromaDB collection keyed by their user ID, and retrieval at generation time is scoped exclusively to that namespace. This enables a single shared model to produce demonstrably personalised outputs for different users concurrently. This approach is architecturally novel and raises a research question that has not been directly addressed in the code generation literature: how much stylistic fidelity can be achieved through prompt-context personalisation alone, on a shared cloud model, without any weight modification?

The Novelty of This Work This component fills these gaps with three distinct contributions absent from the existing literature. First, it provides the first system that automatically extracts and persistently enforces a specific developer's coding conventions from their own repository at inference time, across full multi-file backend projects with automatic cross-file

propagation. Second, it introduces a multi-tenant RAG architecture that guarantees per-user style isolation on a shared cloud model, enabling personalised code generation at scale without weight modification — and empirically quantifies the personalisation gain this achieves. Third, it integrates all of these capabilities within a Human-in-the-Loop multi-agent pipeline with execution-grounded pre-testing validation, exposed as a production REST API — a deployment model that no prior academic system in this domain has adopted.

2. OBJECTIVE

2.1 Main Objective

To design and implement a cloud-deployed, Style-Aware Generative Coding Agent exposed as a REST API that performs Full-Project Synthesis of Node.js/Express and MongoDB backends by automatically extracting the developer's personal coding style from their repository, enforcing it at inference time through a multi-tenant namespaced RAG architecture, and guaranteeing structural integrity and cross-file consistency through episodic memory and pre-testing validation.

2.2 Specific Objectives

1. Develop an AST-based (Tree-sitter) style extraction pipeline combined with a multitenant RAG system (ChromaDB with per-user namespaced collections) to analyse the developer's existing local or GitHub repository, extract latent coding conventions, and build a persistent isolated UserStyle knowledge base per developer.
2. Implement the File System Manager (FSM) using L2MAC-style Control Unit principles adapted for cloud object storage (AWS S3 or GCS) to automatically create and maintain complete multi-file Node.js/Express project structures — including models, controllers, routes, middleware, package.json, and Swagger documentation — without context-window overflow.
3. Design the Logic Injection Mechanism with Reflexion-inspired Episodic Memory and CodeChain-style sub-module clustering to propagate style rules and reusable modules across all interdependent files (models → controllers → routes → documentation → tests), with episodic state persisted in a cloud database per user.
4. Implement a Pre-Testing Syntax and Logic Validation Layer (Self-Edit and SelfDebug style) using esprima parsing, Docker-based ephemeral execution sandboxes, and fault-aware refinement inside the Coding Agent before handover to the Testing Agent.

3. METHODOLOGY

3.1 Research Approach

This component adopts the Design Science Research (DSR) methodology (Hevner et al., 2004), which is particularly suited to research that produces and evaluates innovative computational artefacts in response to real organisational problems [14]. The artefact in question is the Style-Aware Generative Coding Agent deployed as a cloud REST API, and its evaluation spans both quantitative technical metrics and, in later stages, usability testing with volunteer developers from Sri Lanka's IT industry.

Development proceeds iteratively through four cycles: artefact design grounded in the literature review; prototype implementation of each sub-component; quantitative evaluation against defined performance metrics; and refinement through self-revision loops informed by evaluation findings. This iterative structure aligns with the broader Human-in-the-Loop Multi-Agent Framework's incremental integration strategy, in which each agent component is developed and validated independently before system-level integration.

3.2 Technical Methods and Algorithms

Style Extraction and Multi-Tenant RAG Pipeline

The foundation of the personalisation system is AST-based style extraction combined with a multi-tenant retrieval architecture. Tree-sitter is used for language-agnostic AST parsing of the developer's JavaScript and TypeScript repository files, extracting a structured profile of their coding conventions: identifier naming patterns (camelCase vs. snake_case frequency, handler and field prefix conventions), error-handling idioms (try-catch placement, custom error class usage, HTTP status code patterns), middleware chaining preferences (authentication ordering, validation placement), Mongoose schema conventions (field type preferences, index usage, virtual and method definitions), and route organisation patterns (flat vs. nested routers, API versioning strategies). This approach is directly informed by the repository-level context extraction techniques of Shrivastava et al. (2023a, 2023b), adapted specifically for style rather than general code completion [2][3].

The extracted style patterns are embedded using a sentence encoder and stored in ChromaDB in a collection uniquely namespaced to the developer's user ID — `style_{user_id}`. This peruser namespace design is the architectural mechanism through which a shared cloud model achieves per-user personalisation. At generation time, the retriever queries exclusively within the requesting user's collection, injecting only that developer's style examples into the prompt context. This is the cloud-native equivalent of a per-user fine-tuned model, achieved without any weight modification. Lewis et al. (2020) established the theoretical foundation for this kind of retrieval-grounded generation [11]; the contribution here is the multi-tenant namespace extension that makes it viable as a personalisation mechanism at scale. When no user repository is provided, a curated fallback collection containing Mongoose, Express, and JWT documentation serves as generation context, following the DocPrompting approach of Zhou et al. (2023) [7].

File System Manager and Cloud Multi-File Synthesis

Generating a single file in isolation is insufficient for production backend development — the challenge is maintaining coherence across an entire multi-file project without overwhelming the model's context window. The File System Manager (FSM) addresses this using the Control Unit principles established by Holt et al. (2024) in L2MAC [1], adapted here for cloud object storage. Rather than writing files to local disk, the FSM manages project files in AWS S3 or GCS under a per-session prefix — for example, `projects/{user_id}/{session_id}/src/models/user.model.js`. The FSM maintains an in-memory registry of all project files, their content hashes, and a dependency graph derived from import and require statements. When any upstream file is modified, the FSM automatically identifies all dependent files and queues them for context-aware regeneration — implementing automatic cross-file propagation without requiring the entire codebase to occupy the LLM's context window simultaneously. On generation completion, the project is packaged into a downloadable ZIP archive, and a signed URL is returned in the API response.

Logic Injection with Episodic Memory

Generating a style-consistent first draft is necessary but not sufficient — the system must also learn from its own errors and reuse verified sub-modules across future sessions. Drawing directly on the Reflexion framework of Shinn et al. (2023) [4], the Logic Injection Mechanism maintains a per-user episodic memory store persisted in a cloud database. After each generation cycle, any detected style violations — instances where the generated code deviates from the user's extracted conventions — are recorded as verbal reflections. For example: "In the previous session, error responses used `res.status(500).send()` instead of the user's preferred `next(createError(500, message))` pattern — enforce this in all future controller generations." These reflections are retrieved and prepended to subsequent generation prompts, enabling the agent to avoid repeating the same stylistic errors across sessions without any gradient updates to the model.

Complementing this, CodeChain's sub-module clustering approach (Le et al., 2024) [8] is applied to verified generated code. After each project generation, reusable sub-modules — JWT authentication middleware, Mongoose error handlers, request validation schemas, pagination utilities — are extracted, embedded, and clustered using k-means. Cluster centroids, representing the most generic and stylistically consistent implementations, are stored in the episodic memory and injected into future generation contexts. This enables the agent to reuse its own previously verified, style-compliant code rather than generating each module from scratch, progressively building a library of the developer's preferred implementations over time.

Pre-Testing Syntax and Logic Validation

A critical quality gate exists between the Coding Agent and the Testing Agent: generated code must pass syntax and basic logic checks before handover, to avoid wasting expensive testing cycles on trivially broken outputs. The validation pipeline follows the Self-Edit approach of Zhang et al. (2023) [6] and the execution-grounded design of Yang et al. (2023) [10], applied in three sequential stages. Stage 1 uses `esprima` for AST-level JavaScript syntax parsing — any syntactic error triggers a targeted regeneration prompt before proceeding. Stage 2 loads

the generated file into an ephemeral Docker container spun up per request, with mocked Express and Mongoose dependencies, verifying import resolution and catching obvious runtime errors such as undefined variable references and incorrect middleware signatures. Stage 3 applies the rubber-duck debugging technique from Chen et al. (2023) [5]: the agent is prompted to explain the generated code's logic in natural language, identify any inconsistencies with the original JSON blueprint and the user's extracted style conventions, and produce a Self-Edit-style fault-aware refinement using the execution results as supplementary comments. The ephemeral container is destroyed after each validation run, ensuring full execution isolation between concurrent user requests.

3.3 Tools and Platforms

Tool / Platform	Version / Type	Justification
Cloud LLM API (OpenAI / Anthropic / self-hosted DeepSeek via vLLM)	Shared API endpoint	Personalization achieved through per-user RAG context injection; no weight modification required
Tree-sitter	Multi-language parser	Fast, accurate, language-agnostic AST parsing for style extraction from JS/TS repositories
ChromaDB (cloud-hosted or self-hosted)	Vector database	Multi-tenant vector store with per-user name spaced collections ensuring style isolation at query time
FastAPI + LangGraph	Orchestration backend	Agent state management, multi-step workflow control, and inter-agent REST communication
Node.js/Express + Mongoose	Target output stack	Primary technology for generated backends; prevalent in Sri Lankan IT industry
Docker (ephemeral containers) + esprima	Sandbox + JS parser	Per-request ephemeral containers for safe, isolated mock execution; esprima for ASTlevel syntax checking
PostgreSQL / MongoDB (cloud)	Episodic memory store	Persists per-user verbal reflections and sub-module registry across sessions

Table 3.3: Tools and Platforms

4. HIGH-LEVEL SYSTEM ARCHITECTURE

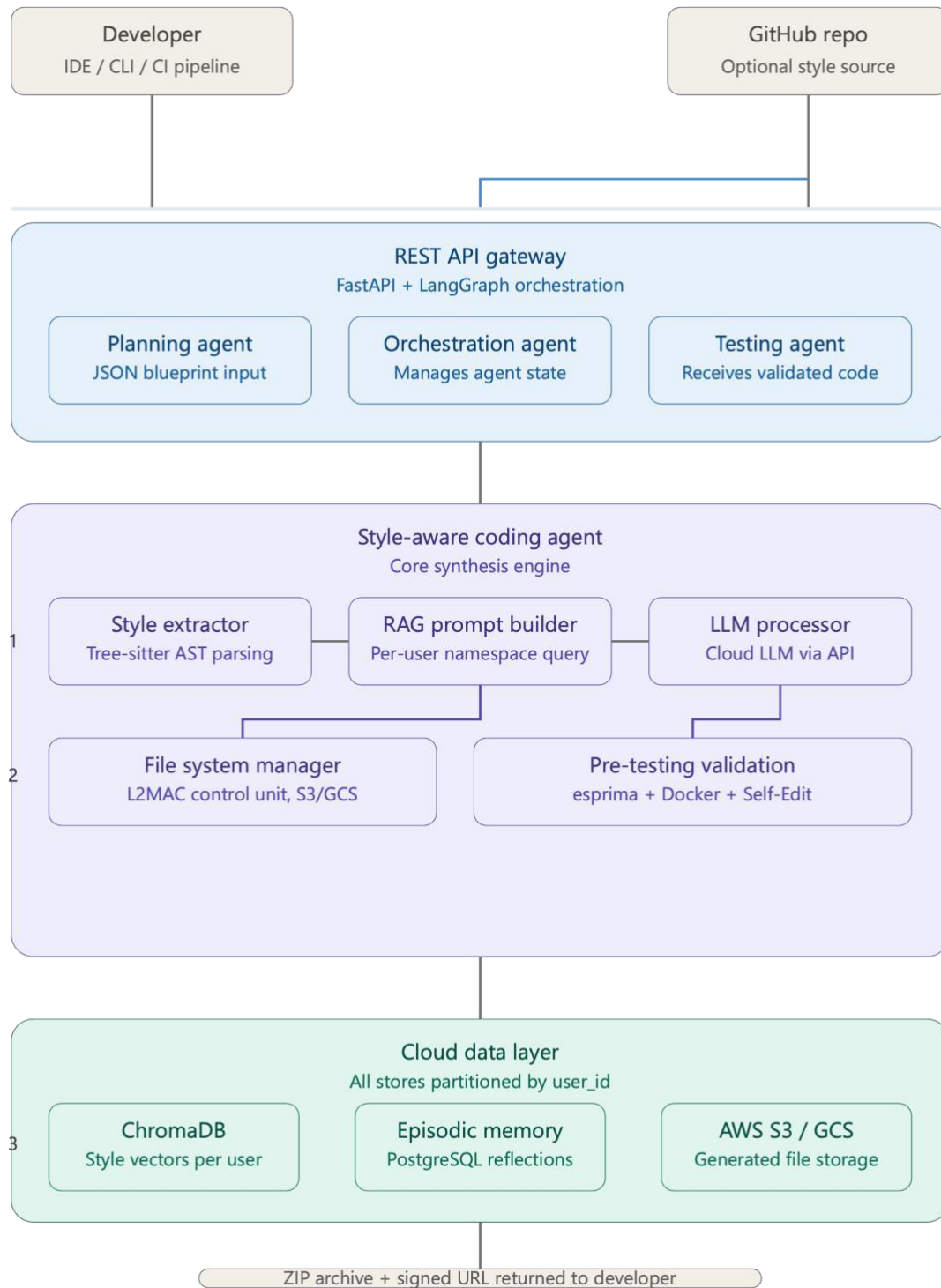


Figure 4.1: System Architecture Diagram

USER REQUIREMENTS

5.1 Stakeholder Identification

The primary stakeholder is the Software Developer or Backend Engineer who interacts with the Coding Agent through the REST API or an IDE integration. Secondary stakeholders include the Project Manager, who benefits from reduced delivery times and lower technical debt, and the QA Team, who receive style-consistent, pre-validated code requiring minimal rework. The system targets individual developers and small-to-medium IT teams in Sri Lanka's software industry who maintain personal or team-specific coding styles across multiple concurrent client projects.

5.2 Functional Requirements

The agent shall accept a structured JSON blueprint from the Planning Agent and an optional path or URL to the developer's existing repository, submitted via a documented REST API endpoint.

The agent shall perform AST-based style extraction from the provided repository using Treesitter and store the extracted conventions in a persistent, per-user namespaced ChromaDB collection isolated from all other users.

The agent shall generate a complete, multi-file Node.js/Express and MongoDB backend (models, controllers, routes, middleware, package.json, Swagger documentation, Jest test scaffolds) while enforcing extracted style conventions across all files through RAG context injection.

The agent shall automatically propagate changes across interdependent files when any upstream file is modified, using the FSM dependency graph and cloud object storage.

The agent shall perform built-in pre-testing validation using esprima syntax checking, ephemeral Docker container mock execution, and Self-Debug rubber-duck refinement before returning code to the Orchestration Agent.

The agent shall maintain a per-user episodic memory of past style violations and reusable sub-modules, persisted in a cloud database and injected into future generation prompts for persistent style learning across sessions.

The agent shall return the completed project as a packaged ZIP archive with a signed download URL in the API response, along with a validation report and Swagger documentation.

The agent shall support a fallback to a curated standard Express and MongoDB style when no user repository is provided.

5.3 Non-Functional Requirements

Performance: Generate a complete backend (5–10 files) in under 3 minutes end-to-end via the REST API on a standard cloud instance (2 vCPU, 4GB RAM API worker).

Usability: Accessible via a documented REST API with clear OpenAPI/Swagger specification; optionally integrable via a VS Code extension or CLI client with no local infrastructure required.

Reliability: Achieve at least 95% structural integrity and at least 90% style consistency across generated files, measured by AST comparison and CrossCodeEval-adapted cross-file checks.

Security and Privacy: All repository data transmitted via TLS 1.3; style embeddings and episodic memory encrypted at rest (AES-256); per-user data namespacing enforced at the storage layer; users may request full data deletion at any time.

Scalability: Support repositories up to 500 files and projects with up to 20 endpoints without context-window issues via L2MAC-style Control Unit management; support concurrent multi-user requests via horizontal scaling.

Maintainability: Modular architecture enabling straightforward addition of new style rule extractors, target frameworks (NestJS, Fastify), or cloud LLM backends in future iterations.

6. COMMERCIALIZATION PLAN

6.1 Target Market and Customer Persona

Primary market: Sri Lankan software development companies (SMEs and freelancing platforms) and individual backend developers maintaining multiple concurrent client projects.
Customer persona: Ravi, a 28-year-old full-stack developer at a Colombo-based startup, maintains 5–7 concurrent client projects, spends 30–40% of his time on repetitive backend code and style refactoring, and wants an API-accessible tool that learns and enforces his personal coding conventions without requiring him to manage any local AI infrastructure.

6.2 Value Proposition

Generates production-ready, style-consistent Node.js backends via a simple API call in minutes rather than hours.

The only tool that automatically learns and persistently enforces a developer's personal coding conventions — commercial tools do not do this.

API-accessible and integrable into any IDE, CLI, or CI/CD pipeline with no local setup required.

Open-source core with a transparent, auditable architecture — unlike the black-box commercial tools.

Multi-tenant architecture means teams can share one deployment while each developer retains their own fully isolated style profile.

6.3 Revenue Model

Open-source core (MIT licence, GitHub) — free for students, individuals, and small teams; builds community adoption and trust.

Premium Enterprise Edition (paid) — custom style profiles, team shared style libraries, VS Code extension with one-click integration, priority support, and SLA guarantees (LKR 2,500–5,000 per month per team).

Freemium API model — limited monthly generation requests free; higher request quotas, multi-project episodic memory, and auto-Swagger with Docker Compose unlocked via subscription.

6.4 Pricing Strategy

One-time individual licence: LKR 15,000. Monthly developer subscription: LKR 1,500. Team bundle (up to 10 developers): LKR 10,000 per month. Training workshops for IT companies: LKR 25,000 per session.

6.5 Competitive Advantage

Unlike Cursor, Replit Agent, and Claude Code — which are style-agnostic and treat all developers identically — this agent learns each developer's personal conventions once from their own repository and enforces them permanently across all future projects via a persistent namespaced RAG profile. The multi-tenant architecture means this personalisation scales to many users on shared infrastructure with guaranteed isolation. This combination is uniquely absent from all commercial and academic tools in the current landscape.

7. BUDGET AND JUSTIFICATION

Resource Category	Estimated Cost
Hardware Costs	20,000
Software License Costs	10,000
Cloud Service Costs	75000
Human Effort Cost	150,000
Total Estimated Budget	255,000

Table 7.1: Budget and Justification Format

The system is built entirely on open-source frameworks. The primary cost during development is LLM API usage for testing, which can be minimised by using the Groq free tier (which serves open models like Llama and DeepSeek at no cost) for initial development and switching to a paid API only for final evaluation.

8. WORK BREAKDOWN STRUCTURE (WBS)

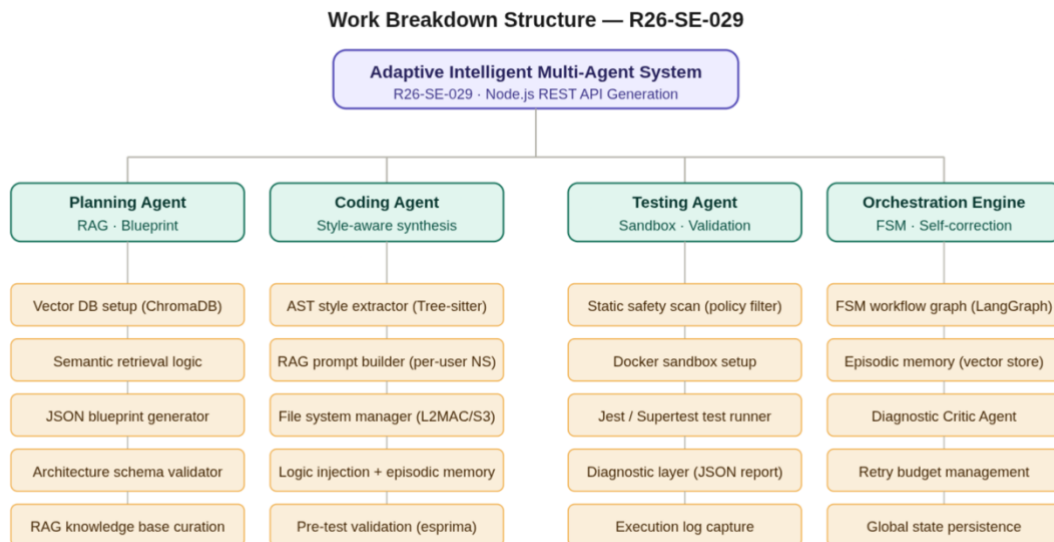


Figure 8.1: Work Breakdown Structure

9. GANTT CHART

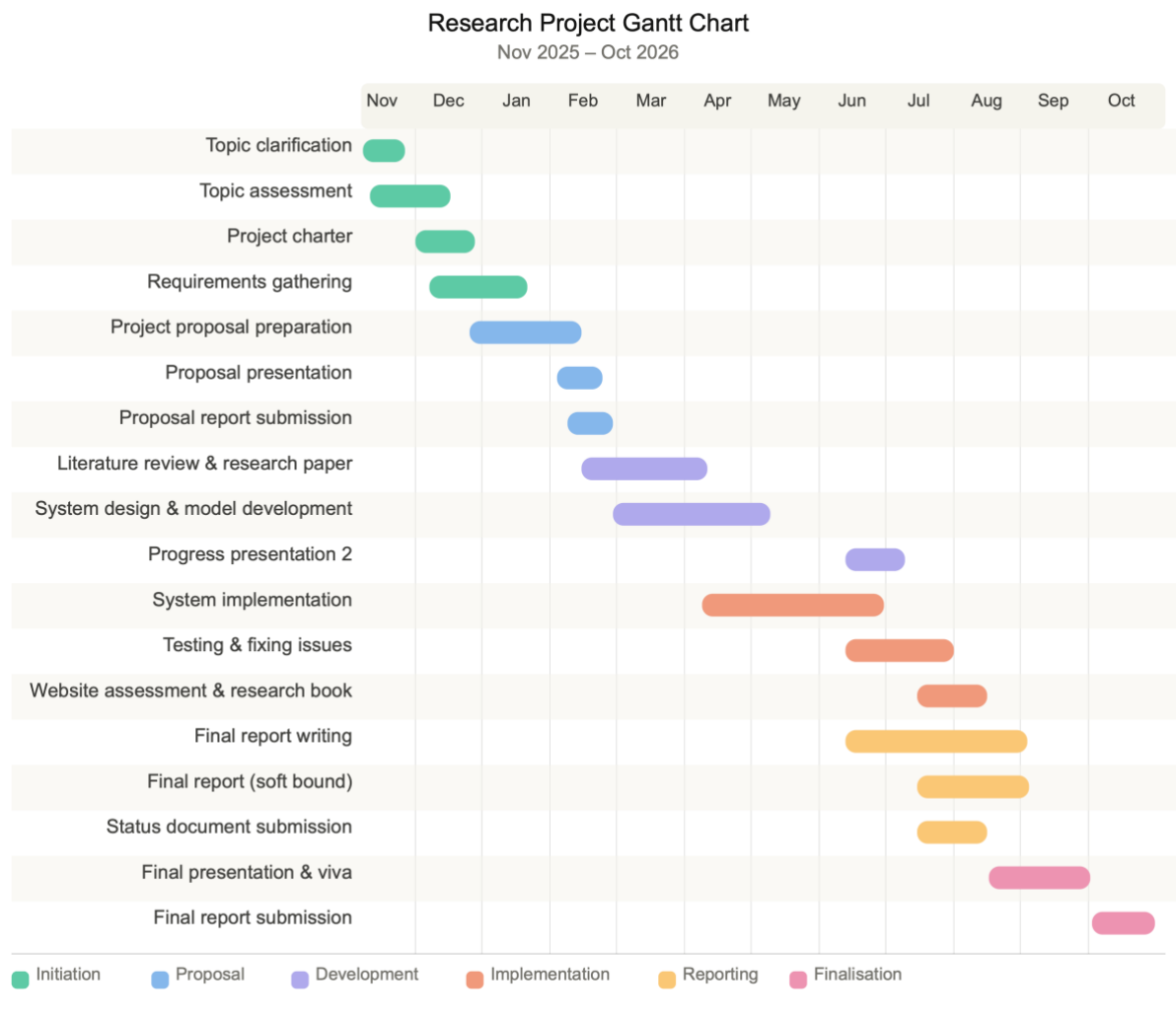


Figure 9.2: Gantt Chart

REFERENCES LIST

- [1] S. Holt, M. Ruiz Luyten, and M. van der Schaar. 2024. "L2MAC: Large Language Model Automatic Computer for Extensive Code Generation." In Proceedings of ICLR 2024, Vienna, Austria. arXiv:2310.02003. <https://arxiv.org/abs/2310.02003>
- [2] D. Shrivastava, D. Kocetkov, H. de Vries, D. Bahdanau, and T. Scholak. 2023. "RepoFusion: Training Code Models to Understand Your Repository." arXiv preprint arXiv:2306.10998. <https://arxiv.org/abs/2306.10998>
- [3] D. Shrivastava, H. Larochelle, and D. Tarlow. 2023. "Repository-Level Prompt Generation for Large Language Models of Code." In Proceedings of ICML 2023, Honolulu, Hawaii, USA. PMLR. <https://proceedings.mlr.press/v202/shrivastava23a.html>
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. 2023. "Reflexion: Language Agents with Verbal Reinforcement Learning." arXiv preprint arXiv:2303.11366. <https://arxiv.org/abs/2303.11366>
- [5] X. Chen, M. Lin, N. Schärli, and D. Zhou. 2023. "Teaching Large Language Models to Self-Debug." arXiv preprint arXiv:2304.05128. <https://arxiv.org/abs/2304.05128>
- [6] K. Zhang, Z. Li, J. Li, G. Li, and Z. Jin. 2023. "Self-Edit: Fault-Aware Code Editor for Code Generation." In Proceedings of ACL 2023, Toronto, Canada. Association for Computational Linguistics. <https://aclanthology.org/2023.acl-long.466/>
- [7] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig. 2023. "DocPrompting: Generating Code by Retrieving the Docs." In Proceedings of ICLR 2023, Kigali, Rwanda. OpenReview. <https://openreview.net/forum?id=ZTCxT2t2Ru>
- [8] H. Le, H. Chen, A. Saha, A. Gokul, D. Sahoo, and S. Joty. 2024. "CodeChain: Towards Modular Code Generation Through Chain of Self-Revisions with Representative Sub-Modules." arXiv preprint arXiv:2310.08992. <https://arxiv.org/abs/2310.08992>
- [9] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang. 2023. "CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion." In Proceedings of NeurIPS 2023, New Orleans, USA. <https://arxiv.org/abs/2310.11248>
- [10] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao. 2023. "InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback." In Proceedings of NeurIPS 2023, New Orleans, USA. <https://arxiv.org/abs/2306.14898>
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In Proceedings of NeurIPS 2020, Vancouver, Canada. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>
- [12] A. S. Ghai, V. Rawat, and K. Ghai. 2024. "Artificial intelligence in system and software engineering for auto code generation." In Proceedings of ICEECT 2024, Dehradun, India. IEEE. <https://ieeexplore.ieee.org/document/10449867>

[13] S. Suri, S. N. Das, K. Singi, K. Dey, V. S. Sharma, and V. Kaulgud. 2023. "Software engineering using autonomous agents: Are we there yet?" In Proceedings of ASE 2023, Luxembourg. IEEE. <https://ieeexplore.ieee.org/document/10298470>

[14] A. R. Hevner, S. T. March, J. Park, and S. Ram. 2004. "Design Science in Information Systems Research." MIS Quarterly, 28(1), 75–105. <https://www.jstor.org/stable/25148625>